

□ PUF-Pay

Hardware-Rooted Payment Authentication Using Silicon Physical Unclonable Functions

GIFT IFIH Young Builders Program — 2026

"Your silicon is your password."

A hardware security IP that generates a unique 128-bit private key from the physical, unclonable variations in a chip's silicon — never stored, always regenerated, impossible to clone.

□ Table of Contents

- [The Problem](#)
- [The Solution](#)
- [How It Works](#)
- [System Architecture](#)
- [Module Breakdown](#)
- [Repository Structure](#)
- [Quickstart — Hardware Simulation](#)
- [Quickstart — Software Demo](#)
- [Demo Walkthrough](#)
- [Security Properties](#)
- [Modeling Notes \(Read This\)](#)
- [Tech Stack](#)
- [Project Status](#)
- [Future Work](#)
- [Author](#)

□ The Problem

Every digital payment today relies on a **secret key stored somewhere**:

- On a SIM card
- In phone memory
- On a bank's server
- Inside a software wallet

If an attacker can **reach that storage location**, they can steal the key and impersonate the user. This is exactly how modern payment fraud happens:

Attack	How It Works
Malware	Reads private key from device memory
SIM Swap	Attacker gets a new SIM, intercepts OTP/keys
Server Breach	Leaked database exposes stored secrets
Chip Cloning	Attacker copies key onto fake hardware

The common root cause: the key exists somewhere it can be found and copied.

The Solution

PUF-Pay never stores the private key at all.

Instead, it is **regenerated from the physical silicon** of the chip every single time a payment is made, then immediately discarded after signing.

This works because of a **Physical Unclonable Function (PUF)**:

Every chip, even from the exact same factory batch, has microscopic manufacturing variations — tiny differences in transistor speed, wire capacitance, and doping levels. These variations are random, permanent, and physically impossible to replicate — even by the original manufacturer.

PUF-Pay measures these variations using **256 ring oscillators** to produce a unique **128-bit fingerprint** for every chip.

No key in memory. Nothing for malware to steal. Nothing to clone.

How It Works

1. ENROLLMENT (one-time, at manufacturing/bank registration)

Chip generates 128-bit key from its own silicon

|



Derives ECDSA public key on-chip

|



ONLY the public key is sent to the bank

(private key never leaves the chip)

2. EVERY PAYMENT

Chip regenerates the SAME 128-bit key from silicon

|



Signs the transaction (ECDSA)

|



Private key is immediately discarded

|



Bank verifies signature using stored public key

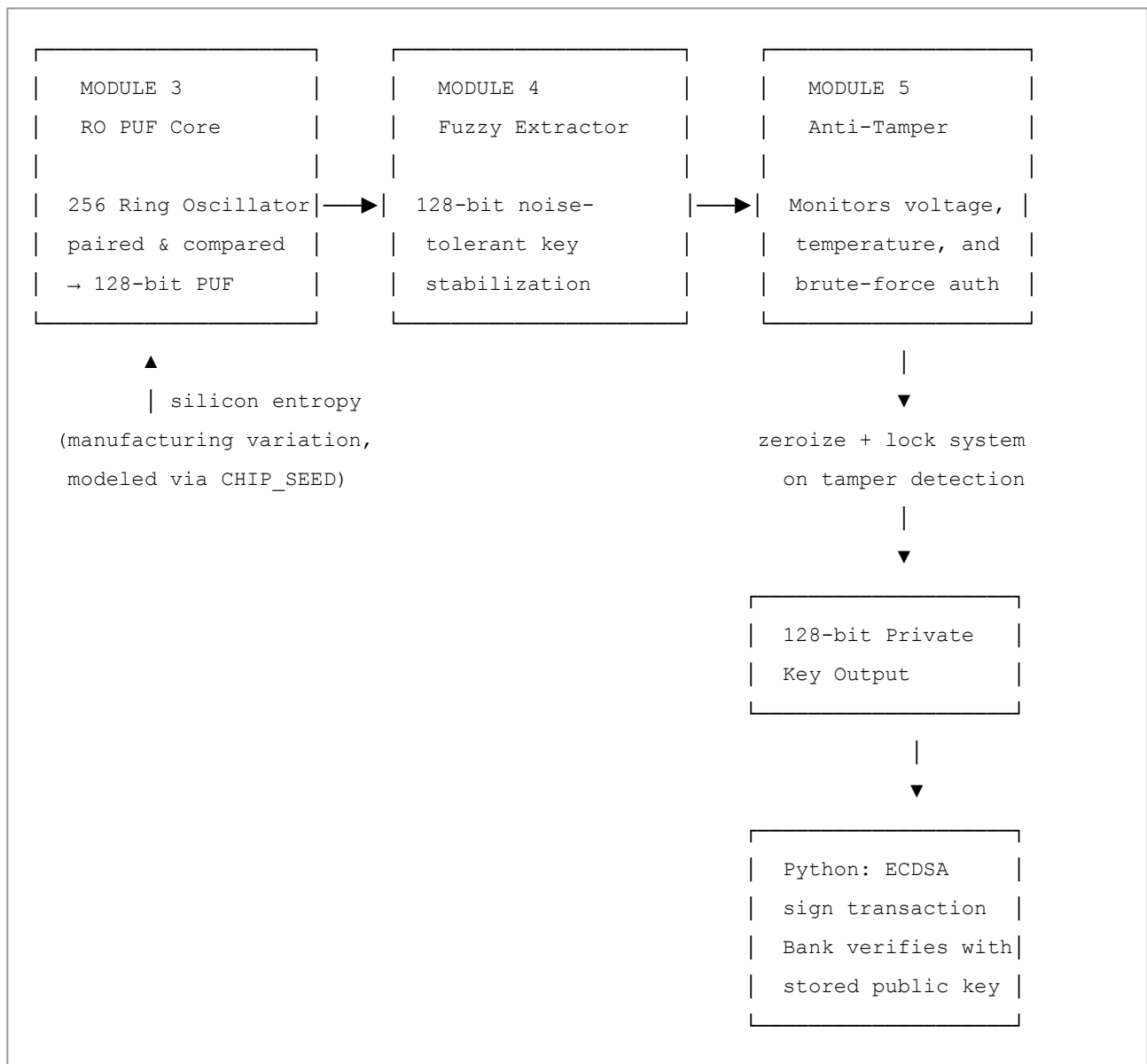
|



☐ Match → Payment Approved

☐ No match → Payment Rejected (different chip / cloned device)

☐ System Architecture



□ Module Breakdown

#	Module	File	Function
1	Ring Oscillator	rtl/ring_oscillator.v	Single odd-stage inverter loop that free-oscillates
2	Frequency Counter	rtl/frequency_counter.v	Counts RO edges in a fixed clock window (with CDC synchronizer)
3	RO PUF Core	rtl/ro_puf_core.v	256 ROs + 256 counters, pairwise frequency comparison → 128-bit PUF response
4	Fuzzy Extractor	rtl/fuzzy_extractor.v	Splits 128-bit response into 16×8-bit blocks, corrects single-bit noise per block
5	Anti-Tamper Monitor	rtl/anti_tamper.v	Detects voltage glitch, temperature attack, brute-force auth failures; zeroizes key + sticky-locks system
6	Top-Level Integration	rtl/puf_pay_top.v	Wires all modules into a single IP block with CHIP_SEED parameter for per-chip uniqueness

Each module has a corresponding testbench in `tb/`, independently verified in Vivado behavioral simulation before full-system integration.

□ Repository Structure

```
PUF-Pay/
├─ rtl/                                # Synthesizable Verilog design files
│   ├── ring_oscillator.v
│   ├── frequency_counter.v
│   ├── ro_puf_core.v
│   ├── fuzzy_extractor.v
│   ├── anti_tamper.v
│   └─ puf_pay_top.v
│
├─ tb/                                # Testbenches (one per module + full system)
│   ├── tb_ring_oscillator.v
│   ├── tb_frequency_counter.v
│   ├── tb_ro_puf_core.v
│   ├── tb_fuzzy_extractor.v
│   ├── tb_anti_tamper.v
│   ├── tb_puf_pay_top.v
│   └─ tb_two_chips.v                # Generates keys for TWO different chips
│
├─ python/                            # Software-side ECDSA signing & verification
│   ├── python_sign.py
│   ├── run_full_demo_v2.py          # □ Main end-to-end demo
│   └─ attack_demo.py
│
├─ shared/                            # Hardware → software bridge (generated keys)
│   ├── puf_key_chipA.txt
│   └─ puf_key_chipB.txt
│
├─ docs/                              # Screenshots, waveforms, diagrams
│
├─ requirements.txt
├─ .gitignore
└─ README.md
```

□ Quickstart — Hardware Simulation (Vivado)

Requirements: Vivado 2025.1, target device xc7a35tcpg236-1 (Artix-7)

1. Create a new Vivado project and add all files from `rtl/` as design sources.
2. Add all files from `tb/` as simulation sources.
3. Set `tb_two_chips.v` as the simulation top module.
4. Run:

```
run 25us
```

5. Observe console output — two 128-bit keys will be generated (Chip A and Chip B), written to `shared/puf_key_chipA.txt` and `puf_key_chipB.txt`.

Expected output:

```
[RESULT] Chip A key = 40c7c9263c0b33f81a830c066d188812
[RESULT] Chip B key = 16ec08203d05a0c001fe5aef391cd1be
[CHECK] PASS □ Different chips produced DIFFERENT keys
```

Individual module testbenches (`tb_ring_oscillator.v`, `tb_frequency_counter.v`, etc.) can also be run standalone to verify each block in isolation.

Important Setup Note

The *testbenches* (`tb_two_chips.v`, `tb_puf_pay_top.v`) currently write generated keys using an **absolute file path** (e.g. `C:/Users/.../shared/puf_key_chipA.txt`), since this repo was developed and tested locally in Vivado on Windows.

Before running simulation on your own machine, open the testbench file and update the `$fopen(...)` path to match your local project location — for example:

```
file = $fopen("YOUR_LOCAL_PATH/shared/puf_key_chipA.txt", "w");
```

Similarly, if you clone this repo and run the Python scripts from a different folder structure, update the file path in `run_full_demo_v2.py` to point to your local `shared/` folder.

This will be replaced with a relative-path solution in a future revision.

Quickstart — Software Demo (Python)

Requirements: Python 3.10+

```
pip install -r requirements.txt
cd python
python run_full_demo_v2.py
```

This reads the two keys generated by Vivado (`shared/puf_key_chipA.txt` and `puf_key_chipB.txt`) and runs the full enrollment → payment → attack story.

Demo Walkthrough

Running `run_full_demo_v2.py` produces this narrative:

```
STEP 1 : Keys read from real RTL hardware simulation
        Chip A key : 40c7c9263c0b33f81a830c066d188812
        Chip B key : 16ec08203d05a0c001fe5aef391cd1be

STEP 2 : Chip A enrolls — sends PUBLIC key to bank
        (private key never leaves the chip)

STEP 3 : Chip A makes a legitimate payment
        Bank verify : ☐ APPROVED

STEP 4 : Chip B (attacker's different/cloned chip) attempts
        the SAME payment using Chip A's registered identity
        Bank verify : ☐ REJECTED
        (different silicon → different key → invalid signature)
```

Zero hardcoded keys. Both Chip A's and Chip B's keys are generated by running the *same* RTL twice with a different `CHIP_SEED` parameter — modeling two physically distinct chips off a manufacturing line.

☐ Security Properties

Attack Vector	PUF-Pay Defense
Key extraction from memory	Key is never stored — regenerated on demand, discarded after use
Chip cloning	Different silicon → different oscillator frequencies → different key
Malware on device	No persistent key exists anywhere for malware to steal
Server / database breach	Server only ever holds the public key
SIM swap fraud	Authentication is bound to chip silicon, not SIM identity
Voltage/clock glitch attacks	Anti-tamper monitor detects out-of-range voltage/temp, zeroizes key, locks system
Brute-force signature attempts	Auto-lockout after 3 consecutive authentication failures
Environmental noise (temp drift)	Fuzzy extractor corrects up to 1-bit error per 8-bit block, keeping key stable

☐ Modeling Notes (Important)

We believe in being transparent about what is *physically real* vs. *simulated* in this prototype:

- **CHIP_SEED parameter:** Real silicon PUFs derive randomness from physical transistor variation, which cannot be modeled in RTL behavioral simulation (all gates behave identically unless told otherwise). We use `CHIP_SEED` combined with a hash-mixing function to inject *deterministic pseudo-randomness* per ring oscillator — this lets us simulate two distinct chips producing genuinely different, non-repeating 128-bit keys. **On real Artix-7 silicon, this entropy would come naturally from manufacturing variation** — no `CHIP_SEED` parameter would exist in the synthesized design.
 - **Enrollment memory in Fuzzy Extractor:** For simplicity, the current `fuzzy_extractor.v` stores the golden PUF response in an internal register during enrollment mode. In a production secure element, only *helper data* (parity bits) would be stored in non-volatile memory — never the raw PUF response itself. This prototype models the concept; production hardening would separate "enrollment memory" into protected secure NVM.
 - **File-based hardware↔software bridge:** Since this project has no physical FPGA board, the 128-bit key is written to a text file by the Vivado testbench and read by Python. In a real SoC, this handoff happens over an internal secure hardware bus in nanoseconds — the file is purely a simulation-to-software bridge for demonstration purposes.
 - **Error correction model:** The fuzzy extractor implements simplified 1-bit-per-block correction (similar in spirit to Hamming codes). A production implementation would use a full BCH(255,131) or similar code for stronger multi-bit error tolerance.
-

□ Tech Stack

- **RTL Design:** Verilog-2001
 - **Simulation:** Xilinx Vivado 2025.1 (behavioral simulation, XSim)
 - **Target Device:** Xilinx Artix-7 (`xc7a35tcpg236-1`)
 - **Cryptography:** Python `ecdsa` library, SECP256k1 curve
 - **Key Expansion:** SHA-256 (128-bit PUF key → 256-bit ECDSA private key)
 - **Bridge:** File-based co-simulation (`shared/*.txt`)
-

□ Project Status

- ☒ Module 1 — Ring Oscillator (verified, oscillation confirmed in waveform)
- ☒ Module 2 — Frequency Counter (verified, CDC-safe edge detection)
- ☒ Module 3 — RO PUF Core, 256 ROs → 128-bit response (verified)
- ☒ Module 4 — Fuzzy Extractor, 128-bit stable key (verified with 0-bit, 1-bit, and 2-bit error injection tests)
- ☒ Module 5 — Anti-Tamper Monitor (verified against voltage, temperature, and brute-force attack scenarios)
- ☒ Module 6 — Full system integration (verified: enrollment → payment → tamper lockout)
- ☒ Two-chip differentiation test (verified: different `CHIP_SEED` → different keys)
- ☒ Python ECDSA signing pipeline (verified end-to-end)
- ☒ Attacker rejection demonstration (verified: cloned chip signature rejected)

Future Work

- Deploy on physical Artix-7 FPGA hardware to capture real silicon entropy
 - Replace simplified error correction with full BCH(255,131) encoder/decoder
 - Move helper-data storage to simulated secure NVM block
 - Add hardware ECDSA accelerator (currently done in software for prototyping)
 - Implement secure JTAG/SPI interface for real hardware↔host communication
 - Statistical PUF quality analysis (inter-chip Hamming distance, bit-aliasing, uniformity)
-

Author

Jay Jain | Siddharth Pandey